



Análisis de operaciones SQL

En relación a las transacciones implementadas, y ejecutando la clase TestClient original (sin modificaciones) se solicita:

- Capturar las operaciones SQL generadas internamente por Hibernate en cada caso de prueba.
- Explicar razonadamente número, el porqué y tipo de operaciones generadas por Hibernate, e indicar pros y contras de las distintas soluciones, relacionándolo con la teoría vista.
- La extensión de dichas explicaciones debe ocupar como máximo dos caras de un folio, sin contar el SQL copiado y pegado de la captura de las trazas.

Este documento con nombre analisis_consultas.pdf se dejará en el subdirectorio .\consultas del proyecto Eclipse entregado.

Análisis de las operaciones

Para empezar en nuestro archivo persistence tenemos que activar la depuración de hibernate, en las propiedades, <property name="hibernate.show_sql" value="true" /> para que imprima todas nuestras sentencias SQL y <property name="hibernate.format_sql" value="true" /> para que podamos tenerlo de una forma mas legible.

Salida de la consulta

Iniciando...

Probando el servicio...

Framework y servicio iniciado...

Insertar compra correcta

Hibernate:

```
select
  cliente0_.cif as cif1_1_0_,
  cliente0_.IDBONOCLIENTE as idbonocliente6_1_0_,
  cliente0_.descripcion as descripcion2_1_0_,
  cliente0_.CP as cp3_1_0_,
  cliente0_.ciudad as ciudad4_1_0_,
  cliente0_.direccion as direccion5_1_0_,
  bonoclient1_.IDBONOCLIENTE as idbonocliente1_0_1_,
  bonoclient1_.bono as bono2_0_1_,
  bonoclient1_.descuento as descuento3_0_1_
from
  HR.Cliente cliente0_
left outer join
  HR.BONOCLIENTE bonoclient1_
  on cliente0_.IDBONOCLIENTE=bonoclient1_.IDBONOCLIENTE
where
  cliente0_.cif=?
Hibernate: select
```



```
menu0_.idMenu as idmenu1_3_0_,
menu0_.descripcion as descripcion2_3_0_,
menu0_.precio as precio3_3_0_
```

```
from
```

```
HR.MENU menu0_
```

```
where
```

```
menu0_.idMenu=?
```

Hibernate:

```
select
```

```
compra0_.CIF as cif1_2_0_,
compra0_.FECHA as fecha2_2_0_,
compra0_.IMPORTE as importe3_2_0_,
compra0_.IDMENU as idmenu5_2_0_,
compra0_.PERSONAS as personas4_2_0_,
cliente1_.cif as cif1_1_1_,
cliente1_.IDBONOCLIENTE as idbonocliente6_1_1_,
cliente1_.descripcion as descripcion2_1_1_,
cliente1_.CP as cp3_1_1_,
cliente1_.ciudad as ciudad4_1_1_,
cliente1_.direccion as direccion5_1_1_,
bonoclient2_.IDBONOCLIENTE as idbonocliente1_0_2_,
bonoclient2_.bono as bono2_0_2_,
bonoclient2_.descuento as descuento3_0_2_,
menu3_.idMenu as idmenu1_3_3_,
menu3_.descripcion as descripcion2_3_3_,
menu3_.precio as precio3_3_3_
```

```
from
```

```
HR.Compra compra0_
```

```
inner join
```

```
HR.Cliente cliente1_
```

```
on compra0_.CIF=cliente1_.cif
```

```
left outer join
```

```
HR.BONOCLIENTE bonoclient2_
```

```
on cliente1_.IDBONOCLIENTE=bonoclient2_.IDBONOCLIENTE
```

```
left outer join
```

```
HR.MENU menu3_
```

```
on compra0_.IDMENU=menu3_.idMenu
```

```
where
```

```
compra0_.CIF=?
```

```
and compra0_.FECHA=?
```

Hibernate:

```
insert
```

```
into
```

```
HR.Compra
```

```
(IMPORTE, IDMENU, PERSONAS, CIF, FECHA)
```

```
values
```

```
(?, ?, ?, ?, ?)
```



OK compra bien insertada

Hibernate:

```
select
  cliente0_.cif as cif1_1_0_,
  cliente0_.IDBONOCLIENTE as idbonocliente6_1_0_,
  cliente0_.descripcion as descripcion2_1_0_,
  cliente0_.CP as cp3_1_0_,
  cliente0_.ciudad as ciudad4_1_0_,
  cliente0_.direccion as direccion5_1_0_,
  bonoclient1_.IDBONOCLIENTE as idbonocliente1_0_1_,
  bonoclient1_.bono as bono2_0_1_,
  bonoclient1_.descuento as descuento3_0_1_
from
  HR.Cliente cliente0_
left outer join
  HR.BONOCLIENTE bonoclient1_
  on cliente0_.IDBONOCLIENTE=bonoclient1_.IDBONOCLIENTE
where
  cliente0_.cif=?
```

Hibernate:

```
select
  menu0_.idMenu as idmenu1_3_0_,
  menu0_.descripcion as descripcion2_3_0_,
  menu0_.precio as precio3_3_0_
from
  HR.MENU menu0_
where
  menu0_.idMenu=?
```

Hibernate:

```
select
  compra0_.CIF as cif1_2_0_,
  compra0_.FECHA as fecha2_2_0_,
  compra0_.IMPORTE as importe3_2_0_,
  compra0_.IDMENU as idmenu5_2_0_,
  compra0_.PERSONAS as personas4_2_0_,
  cliente1_.cif as cif1_1_1_,
  cliente1_.IDBONOCLIENTE as idbonocliente6_1_1_,
  cliente1_.descripcion as descripcion2_1_1_,
  cliente1_.CP as cp3_1_1_,
  cliente1_.ciudad as ciudad4_1_1_,
  cliente1_.direccion as direccion5_1_1_,
  bonoclient2_.IDBONOCLIENTE as idbonocliente1_0_2_,
  bonoclient2_.bono as bono2_0_2_,
  bonoclient2_.descuento as descuento3_0_2_,
  menu3_.idMenu as idmenu1_3_3_,
  menu3_.descripcion as descripcion2_3_3_,
  menu3_.precio as precio3_3_3_
```



```
from
  HR.Compra compra0_
inner join
  HR.Cliente cliente1_
  on compra0_.CIF=cliente1_.cif
left outer join
  HR.BONOCIENTE bonoclient2_
  on cliente1_.IDBONOCIENTE=bonoclient2_.IDBONOCIENTE
left outer join
  HR.MENU menu3_
  on compra0_.IDMENU=menu3_.idMenu
where
  compra0_.CIF=?
  and compra0_.FECHA=?
```

Hibernate:

```
insert
into
  HR.Compra
  (IMPORTE, IDMENU, PERSONAS, CIF, FECHA)
values
  (?, ?, ?, ?, ?)
```

OK compra 2 bien insertada

Insertar compra con fecha nula

OK detecta correctamente que la fecha es nula: Fecha y/o hora incorrecta

Insertar compra con cliente inexistente

Hibernate:

```
select
  cliente0_.cif as cif1_1_0_,
  cliente0_.IDBONOCIENTE as idbonocliente6_1_0_,
  cliente0_.descripcion as descripcion2_1_0_,
  cliente0_.CP as cp3_1_0_,
  cliente0_.ciudad as ciudad4_1_0_,
  cliente0_.direccion as direccion5_1_0_,
  bonoclient1_.IDBONOCIENTE as idbonocliente1_0_1_,
  bonoclient1_.bono as bono2_0_1_,
  bonoclient1_.descuento as descuento3_0_1_
from
  HR.Cliente cliente0_
left outer join
  HR.BONOCIENTE bonoclient1_
  on cliente0_.IDBONOCIENTE=bonoclient1_.IDBONOCIENTE
where
  cliente0_.cif=?
```

OK detecta correctamente que NO existe el cliente

Insertar compra con menú inexistente

Hibernate:

```
select
  cliente0_.cif as cif1_1_0_,
```



```
cliente0_.IDBONOCIENTE as idbonocliente6_1_0_,
cliente0_.descripcion as descripcion2_1_0_,
cliente0_.CP as cp3_1_0_,
cliente0_.ciudad as ciudad4_1_0_,
cliente0_.direccion as direccion5_1_0_,
bonoclient1_.IDBONOCIENTE as idbonocliente1_0_1_,
bonoclient1_.bono as bono2_0_1_,
bonoclient1_.descuento as descuento3_0_1_
from
  HR.Cliente cliente0_
left outer join
  HR.BONOCIENTE bonoclient1_
    on cliente0_.IDBONOCIENTE=bonoclient1_.IDBONOCIENTE
where
  cliente0_.cif=?
```

Hibernate:

```
select
  menu0_.idMenu as idmenu1_3_0_,
  menu0_.descripcion as descripcion2_3_0_,
  menu0_.precio as precio3_3_0_
from
  HR.MENU menu0_
where
  menu0_.idMenu=?
  OK detecta correctamente que NO existe el menú
```

Insertar compra con fecha y cliente existentes

Hibernate:

```
select
  cliente0_.cif as cif1_1_0_,
  cliente0_.IDBONOCIENTE as idbonocliente6_1_0_,
  cliente0_.descripcion as descripcion2_1_0_,
  cliente0_.CP as cp3_1_0_,
  cliente0_.ciudad as ciudad4_1_0_,
  cliente0_.direccion as direccion5_1_0_,
  bonoclient1_.IDBONOCIENTE as idbonocliente1_0_1_,
  bonoclient1_.bono as bono2_0_1_,
  bonoclient1_.descuento as descuento3_0_1_
from
  HR.Cliente cliente0_
left outer join
  HR.BONOCIENTE bonoclient1_
    on cliente0_.IDBONOCIENTE=bonoclient1_.IDBONOCIENTE
where
  cliente0_.cif=?
```

Hibernate:

```
select
  menu0_.idMenu as idmenu1_3_0_,
  menu0_.descripcion as descripcion2_3_0_,
```



```
menu0_.precio as precio3_3_0_  
from  
HR.MENU menu0_  
where  
menu0_.idMenu=?
```

Hibernate:

```
select  
compra0_.CIF as cif1_2_0_,  
compra0_.FECHA as fecha2_2_0_,  
compra0_.IMPORTE as importe3_2_0_,  
compra0_.IDMENU as idmenu5_2_0_,  
compra0_.PERSONAS as personas4_2_0_,  
cliente1_.cif as cif1_1_1_,  
cliente1_.IDBONOCLIENTE as idbonocliente6_1_1_,  
cliente1_.descripcion as descripcion2_1_1_,  
cliente1_.CP as cp3_1_1_,  
cliente1_.ciudad as ciudad4_1_1_,  
cliente1_.direccion as direccion5_1_1_,  
bonoclient2_.IDBONOCLIENTE as idbonocliente1_0_2_,  
bonoclient2_.bono as bono2_0_2_,  
bonoclient2_.descuento as descuento3_0_2_,  
menu3_.idMenu as idmenu1_3_3_,  
menu3_.descripcion as descripcion2_3_3_,  
menu3_.precio as precio3_3_3_  
from  
HR.Compra compra0_  
inner join  
HR.Cliente cliente1_  
on compra0_.CIF=cliente1_.cif  
left outer join  
HR.BONOCLIENTE bonoclient2_  
on cliente1_.IDBONOCLIENTE=bonoclient2_.IDBONOCLIENTE  
left outer join  
HR.MENU menu3_  
on compra0_.IDMENU=menu3_.idMenu  
where  
compra0_.CIF=?  
and compra0_.FECHA=?
```

OK detecta correctamente que existe el cliente y fecha

Insertar Compra con Importe negativo o cero

Hibernate:

```
select  
cliente0_.cif as cif1_1_0_,  
cliente0_.IDBONOCLIENTE as idbonocliente6_1_0_,  
cliente0_.descripcion as descripcion2_1_0_,  
cliente0_.CP as cp3_1_0_,  
cliente0_.ciudad as ciudad4_1_0_,  
cliente0_.direccion as direccion5_1_0_,
```



```
    bonoclient1_.IDBONOCLIENTE as idbonocliente1_0_1_,
    bonoclient1_.bono as bono2_0_1_,
    bonoclient1_.descuento as descuento3_0_1_
from
    HR.Cliente cliente0_
left outer join
    HR.BONOCLIENTE bonoclient1_
    on cliente0_.IDBONOCLIENTE=bonoclient1_.IDBONOCLIENTE
where
    cliente0_.cif=?
```

Hibernate:

```
select
    menu0_.idMenu as idmenu1_3_0_,
    menu0_.descripcion as descripcion2_3_0_,
    menu0_.precio as precio3_3_0_
from
    HR.MENU menu0_
where
    menu0_.idMenu=?
```

Hibernate:

```
select
    compra0_.CIF as cif1_2_0_,
    compra0_.FECHA as fecha2_2_0_,
    compra0_.IMPORTE as importe3_2_0_,
    compra0_.IDMENU as idmenu5_2_0_,
    compra0_.PERSONAS as personas4_2_0_,
    cliente1_.cif as cif1_1_1_,
    cliente1_.IDBONOCLIENTE as idbonocliente6_1_1_,
    cliente1_.descripcion as descripcion2_1_1_,
    cliente1_.CP as cp3_1_1_,
    cliente1_.ciudad as ciudad4_1_1_,
    cliente1_.direccion as direccion5_1_1_,
    bonoclient2_.IDBONOCLIENTE as idbonocliente1_0_2_,
    bonoclient2_.bono as bono2_0_2_,
    bonoclient2_.descuento as descuento3_0_2_,
    menu3_.idMenu as idmenu1_3_3_,
    menu3_.descripcion as descripcion2_3_3_,
    menu3_.precio as precio3_3_3_
from
    HR.Compra compra0_
inner join
    HR.Cliente cliente1_
    on compra0_.CIF=cliente1_.cif
left outer join
    HR.BONOCLIENTE bonoclient2_
    on cliente1_.IDBONOCLIENTE=bonoclient2_.IDBONOCLIENTE
left outer join
    HR.MENU menu3_
```



```
on compra0_.IDMENU=menu3_.idMenu
```

```
where
```

```
compra0_.CIF=?
```

```
and compra0_.FECHA=?
```

OK detecta correctamente que el importe es negativo o cero

Insertar Compra con Personas 0 o negativas

OK detecta correctamente que el número de personas es incorrecto

Quitar descuento del cliente...

Hibernate:

```
select
```

```
cliente0_.cif as cif1_1_0_,
```

```
cliente0_.IDBONOCLIENTE as idbonocliente6_1_0_,
```

```
cliente0_.descripcion as descripcion2_1_0_,
```

```
cliente0_.CP as cp3_1_0_,
```

```
cliente0_.ciudad as ciudad4_1_0_,
```

```
cliente0_.direccion as direccion5_1_0_,
```

```
bonoclient1_.IDBONOCLIENTE as idbonocliente1_0_1_,
```

```
bonoclient1_.bono as bono2_0_1_,
```

```
bonoclient1_.descuento as descuento3_0_1_
```

```
from
```

```
HR.Cliente cliente0_
```

```
left outer join
```

```
HR.BONOCLIENTE bonoclient1_
```

```
on cliente0_.IDBONOCLIENTE=bonoclient1_.IDBONOCLIENTE
```

```
where
```

```
cliente0_.cif=?
```

Hibernate:

```
select
```

```
compras0_.CIF as cif1_2_0_,
```

```
compras0_.FECHA as fecha2_2_0_,
```

```
compras0_.CIF as cif1_2_1_,
```

```
compras0_.FECHA as fecha2_2_1_,
```

```
compras0_.IMPORTE as importe3_2_1_,
```

```
compras0_.IDMENU as idmenu5_2_1_,
```

```
compras0_.PERSONAS as personas4_2_1_,
```

```
menu1_.idMenu as idmenu1_3_2_,
```

```
menu1_.descripcion as descripcion2_3_2_,
```

```
menu1_.precio as precio3_3_2_
```

```
from
```

```
HR.Compra compras0_
```

```
left outer join
```

```
HR.MENU menu1_
```

```
on compras0_.IDMENU=menu1_.idMenu
```

```
where
```

```
compras0_.CIF=?
```

OK valor devuelto por descuento correcto

OK filas correctas en compra después de aplicar descuento



OK Importe en compra correcto después de aplicar descuento

Hibernate:

```
select
  cliente0_.cif as cif1_1_0_,
  cliente0_.IDBONOCLIENTE as idbonocliente6_1_0_,
  cliente0_.descripcion as descripcion2_1_0_,
  cliente0_.CP as cp3_1_0_,
  cliente0_.ciudad as ciudad4_1_0_,
  cliente0_.direccion as direccion5_1_0_,
  bonoclient1_.IDBONOCLIENTE as idbonocliente1_0_1_,
  bonoclient1_.bono as bono2_0_1_,
  bonoclient1_.descuento as descuento3_0_1_
from
  HR.Cliente cliente0_
left outer join
  HR.BONOCLIENTE bonoclient1_
    on cliente0_.IDBONOCLIENTE=bonoclient1_.IDBONOCLIENTE
where
  cliente0_.cif=?
```

Hibernate:

```
select
  compras0_.CIF as cif1_2_0_,
  compras0_.FECHA as fecha2_2_0_,
  compras0_.CIF as cif1_2_1_,
  compras0_.FECHA as fecha2_2_1_,
  compras0_.IMPORTE as importe3_2_1_,
  compras0_.IDMENU as idmenu5_2_1_,
  compras0_.PERSONAS as personas4_2_1_,
  menu1_.idMenu as idmenu1_3_2_,
  menu1_.descripcion as descripcion2_3_2_,
  menu1_.precio as precio3_3_2_
from
  HR.Compra compras0_
left outer join
  HR.MENU menu1_
    on compras0_.IDMENU=menu1_.idMenu
where
  compras0_.CIF=?
```

Hibernate:

```
update
  HR.Compra
set
  IMPORTE=?,
  IDMENU=?,
  PERSONAS=?
where
  CIF=?
```



and FECHA=?

Hibernate:

update

HR.Compra

set

IMPORTE=?,

IDMENU=?,

PERSONAS=?

where

CIF=?

and FECHA=?

Hibernate:

update

HR.Compra

set

IMPORTE=?,

IDMENU=?,

PERSONAS=?

where

CIF=?

and FECHA=?

El valor devuelto es 1242,4938

OK valor devuelto por descuento correcto

OK filas correctas en compra después de aplicar descuento

OK Importe en compra correcto después de aplicar descuento

Quitar descuento con cliente inexistente

Hibernate:

select

cliente0_.cif as cif1_1_0_,

cliente0_.IDBONOCLIENTE as idbonocliente6_1_0_,

cliente0_.descripcion as descripcion2_1_0_,

cliente0_.CP as cp3_1_0_,

cliente0_.ciudad as ciudad4_1_0_,

cliente0_.direccion as direccion5_1_0_,

bonoclient1_.IDBONOCLIENTE as idbonocliente1_0_1_,

bonoclient1_.bono as bono2_0_1_,

bonoclient1_.descuento as descuento3_0_1_

from

HR.Cliente cliente0_

left outer join

HR.BONOCLIENTE bonoclient1_

on cliente0_.IDBONOCLIENTE=bonoclient1_.IDBONOCLIENTE

where

cliente0_.cif=?

OK detecta correctamente que no existe el cliente

Cnosulta con tipo de menú erróneo

Hibernate:

select



```
menu0_.idMenu as idmenu1_3_0_,
compras1_.CIF as cif1_2_1_,
compras1_.FECHA as fecha2_2_1_,
cliente2_.cif as cif1_1_2_,
bonoclient3_.IDBONOCLIENTE as idbonocliente1_0_3_,
menu0_.descripcion as descripcion2_3_0_,
menu0_.precio as precio3_3_0_,
compras1_.IMPORTE as importe3_2_1_,
compras1_.IDMENU as idmenu5_2_1_,
compras1_.PERSONAS as personas4_2_1_,
compras1_.IDMENU as idmenu5_2_0_,
compras1_.CIF as cif1_2_0_,
compras1_.FECHA as fecha2_2_0_,
cliente2_.IDBONOCLIENTE as idbonocliente6_1_2_,
cliente2_.descripcion as descripcion2_1_2_,
cliente2_.CP as cp3_1_2_,
cliente2_.ciudad as ciudad4_1_2_,
cliente2_.direccion as direccion5_1_2_,
bonoclient3_.bono as bono2_0_3_,
bonoclient3_.descuento as descuento3_0_3_
from
  HR.MENU menu0_
left outer join
  HR.Compra compras1_
    on menu0_.idMenu=compras1_.IDMENU
left outer join
  HR.Cliente cliente2_
    on compras1_.CIF=cliente2_.cif
left outer join
  HR.BONOCLIENTE bonoclient3_
    on cliente2_.IDBONOCLIENTE=bonoclient3_.IDBONOCLIENTE
where
  menu0_.idMenu=?
  OK detecta correctamente que NO existe ese menú
```

Información completa con grafos de entidades...

Hibernate:

```
select
  menu0_.idMenu as idmenu1_3_0_,
  compras1_.CIF as cif1_2_1_,
  compras1_.FECHA as fecha2_2_1_,
  cliente2_.cif as cif1_1_2_,
  bonoclient3_.IDBONOCLIENTE as idbonocliente1_0_3_,
  menu0_.descripcion as descripcion2_3_0_,
  menu0_.precio as precio3_3_0_,
  compras1_.IMPORTE as importe3_2_1_,
  compras1_.IDMENU as idmenu5_2_1_,
  compras1_.PERSONAS as personas4_2_1_,
```



```
compras1_.IDMENU as idmenu5_2_0__,
compras1_.CIF as cif1_2_0__,
compras1_.FECHA as fecha2_2_0__,
cliente2_.IDBONOCLIENTE as idbonocliente6_1_2_,
cliente2_.descripcion as descripcion2_1_2_,
cliente2_.CP as cp3_1_2_,
cliente2_.ciudad as ciudad4_1_2_,
cliente2_.direccion as direccion5_1_2_,
bonoclient3_.bono as bono2_0_3_,
bonoclient3_.descuento as descuento3_0_3_
from
  HR.MENU menu0_
left outer join
  HR.Compra compras1_
    on menu0_.idMenu=compras1_.IDMENU
left outer join
  HR.Cliente cliente2_
    on compras1_.CIF=cliente2_.cif
left outer join
  HR.BONOCLIENTE bonoclient3_
    on cliente2_.IDBONOCLIENTE=bonoclient3_.IDBONOCLIENTE
where
  menu0_.idMenu=?
Menu [idMenu=1, descripcion=MENU ESTANDAR, precio=12.5]
  Compra [id=CompraId [fecha=2019-04-13 11:00:00.0, cif=B10000000],
personas=30, importe=375.0]
    Cliente [cif=B10000000, descripcion=Insituto Derby,
direccionPostal=DireccionPostal [direccion=C/Zaragoza 57, CodigoPostal=09003,
ciudad=Burgos], bonoCliente=BonoCliente [idBonoCliente=1]]
      Compra [id=CompraId [fecha=2019-04-12 10:00:00.0, cif=B10000000],
personas=20, importe=250.0]
        Cliente [cif=B10000000, descripcion=Insituto Derby,
direccionPostal=DireccionPostal [direccion=C/Zaragoza 57, CodigoPostal=09003,
ciudad=Burgos], bonoCliente=BonoCliente [idBonoCliente=1]]
          Compra [id=CompraId [fecha=2019-04-15 11:00:00.0, cif=C10000000],
personas=500, importe=5906.25]
            Cliente [cif=C10000000, descripcion=Hospital Palencia,
direccionPostal=DireccionPostal [direccion=C/San Jose 1, CodigoPostal=09003, ciudad=La
Platina], bonoCliente=BonoCliente [idBonoCliente=2]]
              Compra [id=CompraId [fecha=2019-04-14 11:00:00.0, cif=B10000000],
personas=15, importe=187.5]
                Cliente [cif=B10000000, descripcion=Insituto Derby,
direccionPostal=DireccionPostal [direccion=C/Zaragoza 57, CodigoPostal=09003,
ciudad=Burgos], bonoCliente=BonoCliente [idBonoCliente=1]]
OK Sin excepciones en la consulta completa y acceso posterior
FIN.....
```



Descripción de operaciones¹

insertarCompra

En este método se realiza una consulta sobre la tabla cliente contenida en HR, verificando que el cliente exista en la base de datos y que además, sea capaz de recuperar la relacion con la tabla bono cliente, por un left outer join. Esto ocurre porque la relacion @ManyToMany entre Cliente y BonoCliente utiliza carga EAGER por defecto, por lo que Hibernate recupera automáticamente la entidad relacionada mediante un LEFT OUTER JOIN.

En segundo lugar, se realiza otra consulta sobre la tabla menú, en la cual se recupera el mismo por medio de la clave primaria con un simple SELECT. El objetivo principal de esta consulta es comprobar que el menú que se busca existe y recuperar estos datos.

En la tercera consulta tambien tenemos otro SELECT sobre la tabla Compra en la que se comprueba si ya existe una compra con la misma clave primaria. Recordemos que la hemos anidado en otra clase @EmbeddedId (CIF y FECHA), comprobando que no exista otra igual. Podemos ver que por usar el comando find(Compra.class, pk), Hibernate genera algunos SELECT y JOINS con tablas Cliente, Bonocliente y Menú para la comprobación de datos. Esto ocurre principalmente porque todas las @ManyToMany de Compras son de tipo EAGER, por lo que esta comprobación se puede decir que es bastante costosa.

Finalmente, como todas las validaciones se han completado, ejecutamos el insert sobre la tabla compra, indicándonos que se ha hecho con éxito. En caso de detectar algún error el proceso se detiene sin llegar al insert.

Todas estas consultas previas se realizan porque trabajamos con el modelo ORM y se necesita garantizar la integridad referencial antes de persistir datos. El principal inconveniente es que esta cantidad de consultas puede afectar el rendimiento si se necesitan insertar muchos datos.

quitarDescuento

Esta transacción busca al cliente, obtiene sus compras, calcula el nuevo importe sin descuento, modifica cada objeto de la compra, calcula el nuevo precio y luego se hacen los distintos updates.

Primero realiza una SELECT del cliente, donde por medio de un Left outer Join con BonoCliente identifica el bono del mismo. Esto nos permite verificar la existencia del cliente y recuperar los bonos y descuentos asociados.

Luego para recalculer el importe de cada compra hace una SELECT sobre la tabla Compra con otro join hacia la tabla Menu para extraer el precio del mismo.

Luego realiza múltiples UPDATES que actualizan el importe de cada compra. Aparecen varios de estos ya que depende de la cantidad de compras que tenga el cliente. El método recalcula cada importe y luego de realizar los cambios hace el commit.

Según los tesis propuestos, el cliente B1000000 no genera ningún update porque no tiene ningún descuento, el método ve que el importe calculado sin descuento coincide

¹ He creado varios vínculos para referenciar la explicación con el código Hibernate generado anteriormente. Están identificados con colores relacionados con los metodos.



exactamente con el importe almacenado por lo que lo descarta. Sin embargo, el cliente A10000000 sí que tiene 3 compras con descuento por lo que se genera un UPDATE por cada una de las compras modificadas.

List<Menu> consultarMenu

En esta transacción, Hibernate lanza una [consulta SQL mas larga y compleja ya que recupera todos los campos de las tablas Menú, Compra, Cliente y BonoCliente, haciendo joins entre ellas para unir esta información.](#)

Aunque la consulta es más compleja y recupera una mayor cantidad de datos en una sola operación, evita el problema de N + 1 que consiste cuando el framework lanza una consulta para obtener el objeto principal y luego lanza consultas adicionales para poder cargar dependencias. Al usar este grafo como una sola consulta, se trae toda la información de golpe, evitando que al recorrer los resultados se tenga que volver a tener acceso (hacer mas consultas) para buscar datos del cliente reduciendo latencia de la red y el consumo de recursos.

Este grafo lo podemos ver en la clase Menú, que usamos la etiqueta @NamedEntityGraph para indicar que relaciones deben cargarse anticipadamente en esta consulta en concreto. Esto lo aplicamos en el DAO (debido a javax.persistence.loadgraph, haciendo que los campos que no estén en el grafo mantengan el tipo de carga que tengan definido en la entidad (Página 18)).

En nuestra salida de depuración Hibernate podemos ver que la transacción se compone de una [Select principal que incluye todas las tablas unidas por medio de los LEFT OUTER JOINS](#) que permite recuperar todos los datos en una sola sentencia.